

DAS LANGSELECT-PAKET

vo.2.1 langselect

Generierung verschiedener Sprachversionen aus einer Quelle

Matthias WERNER¹

<https://github.com/tuc-osg/osglecture>

Dieses Paket unterstützt die Generierung von Dokumenten in zwei oder drei Sprachvarianten aus einem gemeinsamen Quelldokument. Es wird ein entsprechendes Nuttermakro generiert und es gibt mehrere Methoden, die Zielsprache zu bestimmen.

Das Paket gehört zum osglecture-Bundle, wird in die osglecture-Klasse integriert und vom Buildscript `ollm` unterstützt, kann aber auch eigenständig genutzt werden.

Inhaltsverzeichnis

1	Einleitung	1	3	Nuttermakros	6
1.1	TL;DR	1	3.1	Automatisch Sprachmakros . .	6
1.2	Motivation	2	3.2	Sternvariante	7
1.3	Alternativen	2	3.3	Individuelle Sprachmakros . .	8
1.4	Sprachen	3	4	Interaktion mit anderen Paketen	8
			4.1	Babel/Polyglossia	8
			4.2	Csquotes	9
2	Anwendung	4	5	FAQ	9
2.1	Auswahl der Zielsprache . . .	4	6	Implementation	10
2.2	Optionen	5			

1 Einleitung

1.1 TL;DR

Angenommen Sie wollen von einem Text eine englische und eine deutsche Version erhalten, wobei Sie ein gemeinsames Quelldokument nutzen wollen..

¹ matthias.werner@informatik.tu-chemnitz.de

1. Binden Sie `inlangselect/language/after\usepackage[languages={en,de}]{langselect}`
2. Nutzen Sie im Quelldokument `\lende{⟨Texten⟩}{⟨Textde⟩}`. `⟨Texten⟩` wird in der englischen Version erscheinen, während `Textde` in der deutschen Version erscheinen wird.
 - Text, der in beiden Versionen vorkommt, schreiben Sie außerhalb des Makros oder nutzen die Sternversion, um gemeinsame Daten im Fließtext nur einmal schreiben zu müssen, siehe Abschnitt 3.2.
3. Übersetzen Sie das Dokument mit `um` die englische Version zu erhalten, und mit `langselect/language/after` für die deutsche.


```
> latex "\def\olsTargetLanguage{en}\include{⟨document⟩}"
> latex "\def\olsTargetLanguage{de}\include{⟨document⟩}"
```

 - Es gibt noch einige andere Möglichkeiten, die Zielsprache zu bestimmen, u. a. über den Jobnamen und über `\DocumentMetadata`.

1.2 Motivation

Mitunter werden mehrere Sprachversionen des gleichen Dokuments benötigt. Im Anwendungsgebiet des `osglecture`-Bündel sind dies beispielsweise die Lehrskripte, Vorlesungsfolien und Handouts einer Lehrveranstaltung. Es fällt häufig schwer, mehrere parallel existierende Sprachversionen konsistent zu halten. Diese Aufgabe ist etwas einfacher, wenn alle Versionen in einem gemeinsamen Quelldokument enthalten sind.

Dieses Paket unterstützt die Generierung solcher Sprachversionen aus einem gemeinsamen $\text{\LaTeX}$ -Quelldokument indem es Makros für Sprachvarianten und Mechanismen zur Auswahl der Zielsprache bereitstellt.

1.3 Alternativen

Es gibt eine Reihe von Alternativen zu `langselect`.

Adhoc-Makros Man kann relativ einfach adhoc Lösungen schaffen, wie z. B.

```
1 \newif\ifenglish
2 \englishtrue
3 \ifenglish
4     Welcome!
5 \else
6     ¡Bienvenido!
7 \fi
8
```

Tatsächlich folgt `langselect` grundsätzlich diesem Ansatz, automatisiert aber die Erstellung der Sprachmakros, vereinfacht die gemeinsame Nutzung von nichtsprachabhängigen

Elementen wie z. B. Formeln und ermöglicht die Auswahl der Sprachvariante über verschiedene Methoden.

multilang Das ausgeklügelte Paket [multilang](#)² von Richard GREWE erstellt Sprachversionen von gegebenen Makros, wobei die aktive Sprache direkt aus [Babel](#) oder [Polyglossia](#) übernommen wird. [langselect](#) hat ein ähnliches Ziel, aber ein anderes (aus Sicht des Paketautoren: praktischeres) Interface. Falls Sie aber den Ansatz von [langselect](#) nicht mögen, sollten Sie sich unbedingt [multilang](#) anschauen.

comment Victor EIJKHOUT hat das Paket [comment](#)³ geschrieben, das auf einfache Weise ermöglicht, nur bestimmte Abschnitte im Dokument auszugeben. Dies kann auch sehr gut für eine Sprachauswahl genutzt werden.

bicaption/translations/translator/xt_capt Die Pakete adressieren Programmierer von $\text{\LaTeX}$ -Paketen, um fixe Textelemente wie die Überschriften von Verzeichnissen zu internationalisieren. Für Dokumentautoren sind diese Pakete nur bedingt geeignet. [langselect/language/afterbicaption](#)⁴/[translations](#)⁵/[translator](#)⁶/[xt_capt](#)s⁷

1.4 Sprachen

Das Konzept von [langselect](#) beinhaltet die Nutzung von Sprachen auf verschiedenen Ebenen mit unterschiedlicher Wirkung. Um Verwechslungen zu vermeiden, wollen wir hier die Begriffe eindeutig definieren.

1. Die Menge *möglicher* Zielsprachen des Dokuments. Ein Dokument, das [langselect](#) nutzt, enthält typischerweise Textteile in verschiedenen Zielsprachen. Die möglichen Zielsprachen werden über eine Paketoption gesetzt, siehe Abschnitt Im weiteren Text dieser Dokumentation nennen wir die *möglichen* Zielsprachen *Auswahlsprachen*, da für die $\text{\LaTeX}$ -Übersetzung eine dieser Sprachen ausgewählt wird. [langselect/language/after](#)2.2
2. Die tatsächliche bei einer $\text{\LaTeX}$ -Übersetzung gebrauchte Zielsprache. Diese muss eine der Sprachen aus 1. sein. Typischerweise wird der Übersetzungsvorgang für jede der tatsächlichen Zielsprachen einzeln ausgeführt. Für die Auswahl der tatsächlichen Zielsprache gibt es mehrere Methoden, siehe Abschnitt 2.1.
3. Die Sprache, in der Textteile oder einzelne Wörter tatsächlich geschrieben sind und die beispielsweise bei der Worttrennung berücksichtigt wird. Dies ist das Konzept von Sprache, das Pakete wie [Babel](#) oder [Polyglossia](#) haben.⁸ Bitte beachten Sie, dass ein mehrsprachiges Dokument aus der Sicht von [langselect](#) ein monosprachiges Dokument

2. on CTAN as [multilang](#): <http://mirrors.ctan.org/macros/latex/contrib/multilang/>

3. on CTAN as [comment](#): <http://mirrors.ctan.org/macros/latex/contrib/comment/>

4. on CTAN as [bicaption](#): <http://mirrors.ctan.org/macros/latex/contrib/bicaption/>

5. on CTAN as [translations](#): <http://mirrors.ctan.org/macros/latex/contrib/translations/>

6. on CTAN as [translator](#): <http://mirrors.ctan.org/macros/latex/contrib/translator/>

7. on CTAN as [xt_capt](#)s: http://mirrors.ctan.org/macros/latex/contrib/xtcapt/xt_capt/

8. Wobei auch in diesen Paketen „Sprache“ auch für mehrere unterschiedliche und z. T. voneinander unabhängige Konzepte wie Trennungsmuster, zu nutzende Fonts etc. steht

aus der Sicht von Babel sein kann. Häufig ist es aber eher umgekehrt, so dann durchaus noch weitere Sprachen als die Auswahlssprachen hinzukommen: Beispielsweise kann ein Dokument mit den Auswahlssprachen Deutsch und Englisch im Text lateinische Wörter oder Sätze enthalten. Diese Auswahl wird über die üblichen Mechanismen wie in `\selectlanguage`, `\foreignlanguage` oder `\text{<sprache>}` vorgenommen. Wenn Babel oder Polyglossia geladen wird, was auf Wunsch auch direkt über `langselect` erfolgen kann, wird als aktive Sprache die Zielsprache ausgewählt.

Sprachen werden in `langselect` über ISO 639-1-Codes (z. B. „en“, „fr“, „de“, ...) bezeichnet. Wenn für die Sprachbehandlung auf der Ebene von Babel/Polyglossia ein bestimmter Dialekt oder eine bestimmte Variante gewünscht ist, kann ein Mapping erfolgen.

2 Anwendung

Das Paket wird auf die übliche Weise geladen:

```
\usepackage[<options>]{langselect}
```

Da die Auswahl der Zielsprache sowohl über Optionen als auch auf anderem Weg erfolgen kann, werden hier zunächst diese verschiedenen Wege beschrieben. Weitere Optionen wie das Setzen der Auswahlssprachen werden in Abschnitt `esprochen.langselect/language/after2.2`

2.1 Auswahl der Zielsprache

Um eine Zielsprache für einen $\text{\LaTeX}$ -Übersetzungslauf zu bestimmen, gibt es verschiedene Möglichkeiten. Die Reihenfolge in der folgenden Liste spiegelt auch die Reihenfolge der Auswertung. Sobald damit eine Zielsprache ermittelt wird, wird eine weitere Auswertung abgebrochen.

1. Definition eines Makros `\olsTargetLanguage` vor Laden des Pakets. Damit ist es beispielsweise möglich, über

```
> latexmk -usepretex="\def\olsTargetLanguage{fr}" document.tex
```

die Zielsprache Französisch für den Übersetzungslauf festzulegen.

2. Setzen der Paketoption Verschiedene Werte sind hier möglich `langselect/language/aftertargetlang`:

```
targetlang = <ISO-Code>
```

Die Zielsprache wird direkt durch den Code angegeben.

```
targetlang = {job=<n>}
```

Die Zielsprache wird aus dem n . Element des Jobnamens bestimmt. Ein Element ist ein mit „-“ (Minus) abgetrennter Teil des Jobnamens, siehe die Dokumentation zum Paket `varsfromjobname`,⁹ das dafür geladen wird. Beispielsweise kann bei `targetlang = {job=2}` über

```
> latex -jobname doc-ru doc.tex
```

9. on CTAN as `varsfromjobname`: <http://mirrors.ctan.org/macros/latex/contrib/varsfromjobname/>

Russisch als Zielsprache eingestellt werden.

`targetlang = babel|polyglossia`

Für die Nutzung dieses Schlüsselwerts muss Babel oder Polyglossia vor `langselect` geladen sein. Die Zielsprache wird dann auf die durch das Sprachpaket eingestellte (Haupt-)Sprache gesetzt.¹⁰

`targetlang = meta`

Die in `\DocumentMetadata` angegebene Sprache als Zielsprache verwendet.

3. Wenn weder `\olsTargetLanguage` definiert noch die Option `targetlang` gesetzt wird, versucht `langselect` selbständig die Zielsprache zu bestimmen, indem es erst die Option `targetlang = babel` und dann (wenn kein Sprachpaket gefunden wurde) die Option `targetlang = meta` ausprobiert.
4. Wenn alles andere versagt, wird Englisch als Zielsprache gesetzt.

2.2 Optionen

Folgende Optionen stehen zur Verfügung:

`languages = {⟨Liste von Auswahl Sprachen⟩}` (erforderlich)

Gibt zwei oder drei Auswahl Sprachen an. Die Reihenfolge ist relevant für die automatische Generierung der Sprachmakros (siehe Abschnitt . `langselect/language/after3`)

`targetlang = ⟨Sprache⟩|{job=⟨n⟩}|babel|polyglossia|meta`

Siehe Abschnitt 2.1

`prefix = {⟨prefix⟩}`

Voreinstellung: 1

Setzt einen Prefix für die Multisprachmakros. Es kann auch ein leerer Prefix gesetzt werden.

`auto = true|false`

Voreinstellung: true

Die eigentlichen Sprachmakros werden automatisch angelegt, siehe Abschnitt . Wird `auto = false` gesetzt, werden nur die Makros zur Erzeugung der Sprachmakros zur Verfügung gestellt, nicht die Sprachmakros selbst. `langselect/language/after3`

`trim = true|false`

Voreinstellung: true

Enfernt Whitespaces am Anfang und Ende der Argumente der Sprachmakros.

`load babel = {⟨Babel Optionen⟩}`

`load polyglossia` Polyglossia Optionen Lädt entweder Babel oder Polyglossia so, dass die Zielsprache von `langselect` als Hauptsprache gesetzt wird. Die hier übergebenen Optionen werden an das jeweilige Sprachpaket weitergeleitet. Wenn eine dieser Option genutzt wird, darf nicht `targetlang = babel` oder `targetlang = polyglossia` gesetzt sein und `langselect` muss vor dem jeweiligen Sprachpaket geladen werden.

10. Beide Werte bewirken exakt das Gleiche: Da Babel und Polyglossia die gleiche Schnittstelle anbieten, ist es egal, welches von beiden Paketen geladen ist. `targetlang = polyglossia` findet also auch Babel.

`map = {⟨Sprache⟩=⟨Variante⟩,...}`

Legt ein Mapping von Sprachen zu einer Sprachvariante fest. Wenn `langselect` mit anderen Paketen interagiert, wird dann die spezifische Sprachvariante statt des ISO-Codes weitergegeben.

Falls Sie beispielsweise Babel von `langselect` aus laden und Ihr Dokument explizit *Britisches Englisch* und *Schweizerdeutsch* nutzt, sichern Sie mit folgendem Code ab, dass Babel korrekte Optionen erhält.

```

1 \documentclass{article}
2 \usepackage[
3   languages={en,de},
4   load=babel,
5   map={en=british,de=nswissgerman}
6 ]
7 \begin{document}
8   \title{\lende{Hallo}{Grüezi}}
9 
```

`unified shorthands = true|false`

Voreinstellung: true

Definiert sprachenübergreifende Shorthands zur Unterstützung von `csquotes`.

3 Nutzermakros

3.1 Automatisch Sprachmakros

Wenn Sie nicht die Option `auto = false` gesetzt haben, wird ein Multisprachmakro angelegt. Dessen Name wird durch die Liste der Auswahlssprachen festgelegt. Je nachdem, ob Sie zwei oder mehr Auswahlssprachen angegeben haben, ist ist der Name

- `\⟨Prefix⟩⟨Sprache1⟩⟨Sprache2⟩{⟨Text1⟩}{⟨Text2⟩}`

oder

- `\⟨Prefix⟩⟨Sprache1⟩⟨Sprache2⟩⟨Sprache3⟩{⟨Text1⟩}{⟨Text2⟩}{⟨Text3⟩}`

Beispielsweise für für diese Dokumentation das Paket mit folgenden Optionen geladen:

```

1 \usepackage[
2   languages={de,en},
3   targetlang={job=2}
4 ]{langselect}
```

Damit steht das Makro `\ldeen{⟨Text1⟩}{⟨Text1⟩}` zur Verfügung. Wenn jetzt die Zielsprache „de“ ausgewählt wird (hier über den Jobnamen, aber es gibt auch andere Möglichkeiten, vgl.

Abschnitt , wird nur $\langle Text_1 \rangle$ in das PDF-Dokument übernommen und $\langle Text_2 \rangle$ verworfen. Bei der Zielsprache „en“ ist es umgekehrt. `langselect/language/after2.1`

Achtung!

Die automatische Generierung der Sprachmakros kann manchmal zu Problemen führen. Beispielsweise ergibt Abchasisch (Code: „ab“) als Erst- und Griechisch (Code: „el“) als Zweisprache den automatischen Namen `\label`, was zu Fehlermeldungen führt. Nutzen Sie in solchen Fällen die im Abschnitt 3.3 beschriebenen Möglichkeiten.

3.2 Sternvariante

Ein häufiges Problem bei mehreren Sprachversionen ist die Konsistenz nichtsprachabhängiger Daten, insbesondere wenn diese Daten sich öfter ändern. (Denken Sie z. B. an Stundenpläne.)

Für solche Fälle stellt `langselect` eine Sternvariante der Sprachmakros bereit. In dieser Sternvariante können die Textargumente Platzhalter der Form „@ $\langle n \rangle$ “ enthalten, wobei $\langle n \rangle$ einen Wert zwischen 1 und 9 darstellt. Hinter dem Makro müssen dann weitere Argumente angegeben werden, und zwar so viele, wie es dem höchsten verwendeten $\langle n \rangle$ entspricht. Die Sternvariante ersetzt die Platzhalter mit den entsprechenden Argumenten.

Die Sternvariante der Sprachmakros ist „zerbrechlich“, bitte verwenden Sie sie nicht in einem beweglichen Kontext, z. B. innerhalb eines `\section`-Arguments.

```

1 \documentclass{article}
2 \usepackage[languages={en,fr},
3 targetlang=fr]{langselect}
4
5 \begin{document}
6 \lenfr*{
7   The formula @1 generally refers to the Pythagorean theorem.
8 }{
9   La formule @1 fait généralement référence au théorème de
10  Pythagore.
11 }{ $a^2+b^2=c^2$ }
12
13 \lenfr*{
14   Calculate the length of the third side if one side is @1 units long
15   and the hypotenuse is @2 units long.
16 }{
17   Calculez la longueur du troisième
18   côté si un côté mesure @1 unités et l'hypoténuse @2 unités.
19 }{ $\$4$ }{ $\$5$ }
20 \end{document}
```

La formule ait généralement référence au théorème de Pythagore. langselect/language
 $b^2 = c^2$
 Calculez la longueur du troisième côté si un côté mesure nités et l'hypoténuse
 langselect/language/after unités. 45

3.3 Individuelle Sprachmakros

Wenn Ihnen die generierten Namen der Sprachmakros nicht gefallen oder diese sogar zu Kollisionen führen, können Sie diese ändern. Am einfachsten ist es, die Paketoption `prefix` zu ändern, so dass beispielsweise mit `prefix = mv` das Sprachmakro für die Deutsch/Englisch-Kombination `\mvdeen` heisst.

Wenn dies nicht ausreichend ist, geben Ihnen die folgenden zwei Makros die vollständige Kontrolle über die Namensbildung.

```
\olsMakeBilingualMacro[⟨Prefix⟩]{⟨Sprache1⟩}{⟨Sprache2⟩}[⟨Basisname⟩]
```

```
\olsMakeTrilingualMacro[⟨Prefix⟩]{⟨Sprache1⟩}{⟨Sprache2⟩}{⟨Sprache3⟩}[⟨Basisname⟩]
```

Diese Makros legen ein bilinguals bzw. trilinguals Sprachmakro für die angegebenen Sprachen an. Wenn `⟨Basisname⟩` angegeben wird, ist der Name des Sprachmakros `\⟨Prefix⟩⟨Basisname⟩`. Wenn das `⟨Prefix⟩`-Argument benutzt wird, bestimmt dieses den Prefix, sonst gelten die Paketoptionen.

4 Interaktion mit anderen Paketen

4.1 Babel/Polyglossia

Das Paket `langselect` ist zunächst einmal unabhängig von Sprachpaketen wie Babel oder Polyglossia. Es kann ohne oder neben diesen Paketen eingesetzt werden. Wenn man sich jedoch die Mühe eines mehrsprachigen Dokuments macht, will man in der Regel auch, dass für die jeweilige Sprache die richtigen Trennungsregeln, Fonteneinstellungen, Datumsformate etc. benutzt werden.¹¹

Bitte beachten Sie, dass, solange Sie nicht *innerhalb* eines Textes eine andere Sprache (vielleicht ja die Sprache der anderen Sprachversion) nutzen, ihr Dokument aus der Sicht von Babel ein *einsprachiges* Dokument ist. Es besteht die Gefahr, dass Sie Babel auf die gewohnte Weise mit den verschiedenen Sprachen ihrer Sprachversion aufrufen, aber zur Bestimmung der Zielsprache nicht die Option `targetlang = babel` nutzen. Als Ergebnis stimmt u.U. die Zielsprache nicht mit Babels aktiver Sprache überein, und können Sie inkorrekte Trennungen und ähnliche Fehler erhalten.

Um solche Probleme zu vermeiden, bieten `langselect` verschiedene Möglichkeiten:

11. Im Folgenden werde ich mich ausschließlich auf Babel beziehen. Das Gesagte gilt jedoch entsprechend auch für Polyglossia.

1. Wie oben erwähnt kann man die Paketooption `targetlang = babel` nutzen. Dann wird die Zielsprache durch die Hauptsprache von Babel bestimmt. Allerdings muss man dann auf die Möglichkeit verzichten, die Zielsprache „von außen“ (also mit `\olsTargetLanguage` oder über den Jobnamen) zu steuern.
2. Umgekehrt lädt `langselect` Babel oder Polyglossia mit Hilfe der Optionen `load babel/` `load polyglossia` landen und setzt die Zielsprache als Hauptsprache. Weitere Optionen an die Sprachpakete werden als Optionsargument übergeben.
3. Schließlich kann mit dem Hooks `langselect/language/before` und `langselect/language/after` beliebiger Code vor bzw. nach Ausgabe der Argumente des Sprachmakros eingefügt werden, z. B. kann hier `\foreignlanguage` genutzt werden. Der before-Hook hat als Parameter die Zielsprachenvariante.

4.2 Csquotes

Unterschiedliche Sprachen haben unterschiedliche Konventionen zur Setzung von Satzzeichen. Besonders stört das bei Anführungszeichen, da diese der Zielsprache folgen sollen, aber durchaus auch außerhalb der Sprachmakros oder in Argumenten der Sternvariante stehen können. Das Paket `csquotes`¹² bietet dafür eine Lösung, indem es `\enquote` und andere Makros anbietet, die der Zitatkonvention der aktuellen Sprache folgen.

Zur Erhöhung der Bequemlichkeit unterstützt `langselect` die Nutzung korrekter Anführungszeichen mit sprachunabhängigen Shorthands: Wenn die Option `unified shorthands` wahr ist, wird `csquotes` geladen und die Shorthands `"`!` und `"'` auf `\openautoquote` bzw. `\closeautoquote` gesetzt.

Damit werden diese Shorthands in Sprachen wie Deutsch, wo sie bereits definiert sind, überschrieben.

5 FAQ

• Ist `langselect` noch experimentell?

Auf jeden Fall. Während es beim Autoren zwar im täglichen Einsatz ist, ist nicht auszuschließen, dass es unerwünschte Wechselwirkungen zu anderen Paketen gibt. Die Versionsnummer zeigt dies an.

• Wofür steht das „o“ im `\ols`-Prefix??

Das Paket gehört zu Bundle `osglecture`, von dort ist das „o“ geerbt.

• Gibt es Anwendungsbeispiele, die über ein Toy-Hello-World hinausgehen?

Ja, einige. Gucken Sie doch mal zum Beispiel in den Quelltext dieser Dokumentation.

• Ich habe ein @1 in der Ausgabe, wo doch eine Argumentersetzung stattfinden sollte.

12. on CTAN as `csquotes`: <http://mirrors.ctan.org/macros/latex/contrib/csquotes/>

Das hat in der Regel eine der beiden folgenden Ursache:

1. Sie haben den Stern im Sprachmakro vergessen.
2. Sie haben komische Dinge mit Catcodes angestellt. Meist wurde vergessen, ein `\makeatletter` innerhalb der document-Umgebung durch ein `\makeatother` wieder aufzuheben.

- **Kann ich in der Sternversion einen $@n$ -Parameter in einer Sprachversion auslassen?**

Nein. Außerdem wäre dann das Argument ja auch offensichtlich nicht mehr gemeinsam.

- **Kann ich $@n$ -Parameter in gemischter Reihenfolge oder mehr als einmal verwenden?**

Ja.

- **Ich möchte $@$ in einer Sternvariante des Sprachmacros nicht als Parameter verwenden.**

Dann machen Sie es.

- **Aber ich brauche $@42$.**

Nutzen Sie `@{ }42`

- **Ich nutze Russisch, aber der Font ist nicht richtig.**

Das ist keine Problem von `langselect` (das kümmert sich gar nicht um den Font), sondern wahrscheinlich von Babel, `fontenc`, `fontspec`, etc. Aber vielleicht können die Hooks für Sie nützlich sein, sihe Abschnitt 4.1.

- **Meine Frage erscheint hier nicht.**

Sie können Sie unter `tellen.langselect/language/after` <https://github.com/werner-matthias/osglecture/issues>

6 Implementation

```

1 \NeedsTeXFormat{LaTeX2e}[2022/06/01]
2 \ProvidesExplPackage{\packagename}{\packagedate}{\packageversion}{support for different
3 languages versions from a common source}

```

`\olsIsoTags` enthält alle gültigen ISO 639-1-Codestags.

```

4 \seq_const_from_clist:cn{\olsIsoTags}{aa,ab,ae,af,ak,am,an,ar,as,av,ay,az,ba,be,
5   bg,bh,bi,bm,bn,bo,br,bs,ca,ce,ch,co,cr,cs,cu,cv,cy,da,de,dv,dz,ee,el,en,eo,es,
6   et,eu,fa,ff,fi,fj,fo,fr,fy,ga,gd,gl,gn,gu,gv,ha,he,hi,ho,hr,ht,hu,hy,hz,ia,id,
7   ie,ig,ii,ik,io,is,it,iu,ja,jv,ka,kg,ki,kj,kl,km,kn,ko,kr,ks,ku,kv,kw,ky,la,
8   lb,lg,li,ln,lo,lt,lu,lv,mg,mh,mi,mk,ml,mn,mr,ms,mt,my,na,nb,nd,ne,ng,nl,nn,no,
9   nr,nv,ny,oc,oj,om,or,os,pa,pi,pl,ps,pt,qu,rm,rn,ro,ru,rw,sa,sc,sd,se,sg,si,sk,

```

6 Implementation

```
10   sl,sm,sn,so,sq,sr,ss,st,su,sv,sw,ta,te,tg,th,ti,tk,tl,tn,to,tr,ts,tt,tw,ty,ug,
11   uk,ur,uz,ve,vi,vo,wa,wo,xh,yi,yo,za,zh,zu}
12
13   \newcommand\IfTagIsValidF[2]{%
14     \seq_if_in:ceF{olsIsoTags}{#1}{#2}
15   }
16   \RequirePackage{varsfromjobname}[2025/08/03]
```

Das Ergebnisse von `\varsfromjobname` und `\str_range:Nnn` haben Catcode 12, aber Iso-Tags und Optionen haben Catcode 11. Wir vereinheitlichen durch einen Wrapper auf Catcode 11.

```
17   \newcommand\olsChangeCatEleven[1]{
18     \def\tmpa{#1}
19     \edef\tmpb{
20       \noexpand\scantokens{
21         \noexpand\edef\noexpand\olsCatElven{\tmpa}
22       }
23     }
24     \tmpb
25   }
```

Bei Bedarf bilden wir Spachcode auf Sprachvarianten ab. Bei Interaktionen mit externen Code wird dann die Sprachvariante weitergegeben.

```
26   \cs_new:Npn \ols_use_map:n #1 {
27     \cs_if_exist_use:cF { olslangmap / #1 }{ #1 }
28   }
```

Wir definieren einige Optionen. Alle außer `languages` sind optional. Einige Ergebnisse werden wir in Variablen speichern, die wir zuerst anlegen.

```
29   \bool_new:N\ols_load_babel
30   \bool_new:N\ols_load_polyglossia
31   \DeclareKeys{
32     languages.clist_gset:N = \ols_langs,
33     languages.usage=load,
34     targetlang.code = {
```

Die Option `ird` nur ausgewertet, wenn die Zielsprache nicht bereits durch Definition von `langselect/language/after` gesetzt wurde. `targetlang\olsTargetLanguage`.

```
35     \cs_if_exist:NF\olsTargetLanguage{
```

Wird ein Element des Jobnames angegeben?

```
36     \bool_if:nT{
37       \str_if_eq_p:ee{\str_range:nnn{#1}{1}{3}}{job}
38     }{%
```

6 Implementation

```
39     \olsChangeCatEleven{\getfromjobname{\str_range:Nnn{#1}{-1}{-1}}}
40     \xdef\olsTargetLanguage{\olsCatEleven}
41 }
```

Soll die Zielsprache von der langselect/language/after abgeleitet werden `babelpolyglossia`?

```
42     \bool_if:nT{
43       \str_if_eq_p:nn{#1}{babel} || \str_if_eq_p:nn{#1}{polyglossia}
44     }{
45       \str_if_empty:ef{\BCPdata{main.language}}{
46         \edef\olsTargetLanguage{\BCPdata{main.language}}
47       }
48     }
```

Soll die Sprache aus dem Metadaten genutzt werden?

```
49     \str_if_eq:nnTF{#1}{meta}{
50       \IfDocumentMetadataTF{
51         \edef\tmpa{\GetDocumentProperty{document/lang}}
52         \edef\tmpb{\str_range:Nnn\tmpa{1}{2}}
53         \olsChangeCatEleven{\tmpb}
54         \edef\olsTargetLanguage{\olsCatEleven}
55       }{
56         \PackageWarningNoLine{\packagename}{No~meta~data~provided}
57       }
58     }
59 }
```

Wenn bisher noch keine Sprache zugewiesen wurde, wird das Argument als Sprache angenommen. Es wird später überprüft, ob es gültig ist.

```
60     \cs_if_exist:NF \olsTargetLanguage {
61       \edef\olsTargetLanguage{#1}
62     }
63 },
```

Prefix für die Nutzer-Sprachbefehle.

```
64     targetlang.usage=load,
65     prefix.store=\ols_sprefix,
66     prefix.initial:n=l,
67     prefix.usage=load,
```

Automatische Erzeugung der Sprachbefehle?

```
68     auto.bool_set:N=\ols_generate,
69     auto.usage=load,
70     auto.initial:n=true,
```

Automatische Entfernung von Randleerzeichen?

```
71 trim.bool_set:N=\ols_trim,
72 trim.usage=load,
73 trim.initial:n=true,
```

Abbildung auf Sprachvarianten.

```
74 map.code:n = {
75   \prop_new:N \ols_lang_pl
76   \prop_clear:N \ols_lang_pl
77   \prop_set_from_keyval:Nn \ols_lang_pl {#1}
```

Da Property-Listen nicht expandierbar sind, setzen wir die Einträge auf gewöhnlich Makros um.

```
78   \prop_map_inline:Nn \ols_lang_pl
79   { \cs_gset:cpn { olslangmap / \tl_to_str:n {##1} } {##2} }
80   },
```

Laden von @1 oder polyglossia mit Optionenbabelpolyglossia.

```
81 load~babel .code:n = {
82   \bool_set_true:N \ols_load_babel
83   \PassOptionsToPackage{#1}{babel}
84 },
85 load~polyglossia .code:n = {
86   \bool_set_true:N \ols_load_polyglossia
87   \PassOptionsToPackage{#1}{polyglossia}
88 },
89 load.usage = load,
90 unified~shorthands.bool_set:N=\ols_shorthands,
91 unified~shorthands.initial:n=true,
92 unified~shorthands.usage=load
93 }%
94 \ProcessKeyOptions\relax
```

Wenn über `\olsTargetLanguage` oder die `targetlang` keine Zielsprache zugewiesen wurde, untersuchen wir noch einmal, ob über Sprachpakete oder `\DocumentMetadata` eine Sprache identifiziert werden kann.

```
95 \cs_if_exist:NF \olsTargetLanguage {
96   \str_if_eq:eeF{\BCPdata{main.language}}{\edef\olsTargetLanguage{\BCPdata{main.language}}}
97 }
98 \cs_if_exist:NF \olsTargetLanguage {
99   \IfDocumentMetadataT{
100     \edef\tmpa{\GetDocumentProperty{document/lang}}
```

6 Implementation

```
101 \edef\tmpb{\str_range:Nnn\tmpa{1}{2}}
102 \olsChangeCatEleven{\tmpb}
103 \edef\olsTargetLanguage{\olsCatEleven}
104 }
105 }
106 \cs_if_exist:NF \olsTargetLanguage {
107   \PackageWarningNoLine{\packagename}{Can't~identify~any~target~language.
108     \MessageBreak Falling-back-to~'en'}
109   \def\olsTargetLanguage{en}
110 }
```

Wir führen verschiedene Checks durch. Als erstes prüfen wir, ob eine valide Zielsprache ermittelt werden konnte.

```
111 \IfTagIsValidF{\olsTargetLanguage}{
112   \PackageWarningNoLine{\packagename}{Do~not~recognize~language~'\olsTargetLanguage'.
113     \MessageBreak Take~'en'~as~replacement}
114   \def\olsTargetLanguage{en}
115 }
```

Sind mindestens zwei Auswahlssprachen angegeben?

```
116 \int_compare:nNtT{\clist_count:N\ols_langs} < {2}{
117   \PackageError{\packagename}{No~sufficient~number~of~selectable
118     \MessageBreak languages~provided}{
119     You~have~to~provide~a~list~of~two~or~three~\MessageBreak
120     languages~via~'language'~option.}
121   \aftergroup\endinput
122 }
```

Im Moment kann `langselect` nicht mehr als drei Auswahlssprachen verarbeiten. Wir warnen daher, wenn mehr als drei Sprachen angegeben werden.

```
123 \int_compare:nNtT{\clist_count:N\ols_langs} > {3}{
124   \PackageWarningNoLine{\packagename}{Too~many~selectable~languages~provided.
125     \MessageBreak I~will~ignore~the~superflous~languages}
126 }
```

Als nächstes überprüfen wir, ob alle Auswahlssprachen gültige ISO 639 1 Codes sind.

```
127 \clist_map_inline:Nn\ols_langs {
128   \IfTagIsValidF{#1}{
129     \PackageError{\packagename}{Couldn't~resolve~selectable~language~'#1'}{
130       Use~valid~ISO-639-1~code~in~option~'languages'.}
131     \aftergroup\endinput
132   }
133 }
```

Die Zielsprache sollte in der Liste von Auswahlssprachen enthalten sein.

```

134 \clist_if_in:NVF\ols_langs{\olsTargetLanguage}{
135   \PackageError{\packagename}{Target~language~'\olsTargetLanguage'~is~not~\MessageBreak
136     in~the~list~of~selectable~languages}{
137     Check~option~'language'~and~'targetlang'.}
138 }

```

Babel und Polyglossia können nicht beiden geladen werden.

```

139 \bool_if:nT{
140   \ols_load_babel && \ols_load_polyglossia
141 }{
142   \bool_set_false:N \ols_load_babel
143   \bool_set_false:N \ols_load_polyglossia
144   \PackageError{\packagename}{The~options~'load~babel'~and~'load~polyglossia'
145     \MessageBreak are~mutual~exclusive,~you~can't~use~both}{Use~one~load~option,~only.}
146 }
147 \bool_if:nT{
148   ( \cs_if_exist_p:c{ver@babel.sty} || \cs_if_exist_p:c{ver@polyglossia.sty} )
149   &&
150   ( \ols_load_babel && \ols_load_polyglossia )
151 }{
152   \bool_set_false:N \ols_load_babel
153   \bool_set_false:N \ols_load_polyglossia
154   \PackageWarningNoLine{\packagename}{Can't~load~babel/polyglossia~for
155     ~you,since~\MessageBreak
156     a~language~package~is~already~loaded.\MessageBreak
157     I'll~ignore~that~option}
158 }

```

st die Bezeichnung der Zielsprache, wie sie ggf. an externen Code weitergegeben wird, d.,h.

`\DocumentMetadata`, Sprachpakete und Hooks.`langselect/language/after\ols_mapped_target_lang`

```

159 \edef\ols_mapped_target_lang{\ols_use_map:n{\olsTargetLanguage}}

```

Jetzt sollte die Zielsprache feststehen. Wir geben die Informationen ins Log.

```

160 \PackageNoteNoLine{\packagename}{Target~language~is~'\olsTargetLanguage'~(\ols_mapped_target_lang)}

```

Wir korrigieren noch ggf. die über `\DocumentMetadata` gesetzten Metadaten auf die der Zielsprache, so dass beispielsweise Babel keine falsche Hauptsprache ableitet.

```

161 \IfDocumentMetadataT{
162   \edef\olsDocLang{\exp_args:Nc\str_range:nnn{\GetDocumentProperty{document/lang}}{1}{2}}
163   \bool_if:nF{
164     \str_if_eq_p:ee{\olsDocLang}{\olsTargetLanguage}

```

6 Implementation

```
165     }{
166     \PackageWarningNoLine{\packagename}{Target~language~'\olsTargetLanguage'~
167     doesn't~comply~with~\MessageBreak \c_backslash_str
168     DocumentMetadata~'\GetDocumentProperty{document/lang}'.\MessageBreak
169     I'll~try~to~overwrite~the~meta~data}
170     \DocumentMetadata{lang=\ols_mapped_target_lang}
171   }
172 }
```

Da der Name des Sprachmakros nicht bekannt ist, sind Patchen oder generische Hooks komplizierter. Wir stellen deshalb eigene Hooks zur Verfügung.

```
173 \NewHookWithArguments{langselect/language/before}{1}
174 \NewReversedHook{langselect/language/after}
175 \NewHookWithArguments{langselect/argument}{2}
```

Ein Boolean zur Markierung, ob mehr als zwei Auswahlssprachen angegeben wurden.

```
176 \bool_new:c{ols_trilang}
177 \int_compare:nNnTF{\clist_count:N{\ols_langs}} > {2}{
178   \bool_set_true:c{ols_trilang}
179 }{
180   \bool_set_false:c{ols_trilang}
181 }
182 \def\olsFrstLanguage{\clist_item:Nn \ols_langs{1}}
183 \def\olsScndLanguage{\clist_item:Nn \ols_langs{2}}
184 \bool_if:cT{ols_trilang}{
185   \def\olsThrdLanguage{\clist_item:Nn \ols_langs{3}}
186 }
187 \bool_if:NnTF{ols_trim{
188   \let\ols_trim:n=\tl_trim_spaces:n
189 }}{
190   \let\ols_trim:n=\relax
191 }
```

Die Makros `\olsMakeBilingualMacro` bzw. `\olsMakeTrilingualMacro` dienen zur Generierung der eigentlichen Sprachmakros.

```
192 % #1: prefix, #2,#3: selectable languages, #4: base name
193 \NewDocumentCommand{\olsMakeBilingualMacro}{0{\ols_sprefix} m m o}{
194   \str_if_eq:eeT{\olsTargetLanguage}{#2} {
195     \IfValueTF{#4}{
196       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#4}{s +m +m}{
197         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
198         \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##2}}}{\ols_trim:n{##2}}
199         \UseHook{langselect/language/after}
```


6 Implementation

```

200     }
201   }{
202     \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3}{s +m +m}{
203       \UseHook{langselect/language/before}{\ols_mapped_target_lang}
204       \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##2}}}{\ols_trim:n{##2}}
205       \UseHook{langselect/language/after}
206     }
207   }
208 }
209 \str_if_eq:eeT{\olsTargetLanguage}{#3}{
210   \IfValueTF{#4}{
211     \ExpandArgs{c}\NewExpandableDocumentCommand{#1#4}{s +m +m}{
212       \UseHook{langselect/language/before}{\ols_mapped_target_lang}
213       \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##3}}}{\ols_trim:n{##3}}
214       \UseHook{langselect/language/after}
215     }
216   }{
217     \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3}{s +m +m}{
218       \UseHook{langselect/language/before}{\ols_mapped_target_lang}
219       \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##3}}}{\ols_trim:n{##3}}
220       \UseHook{langselect/language/after}
221     }
222   }
223 }
224 }

```

Nun das Gleiche nochmal für *drei* Sprachen.

```

225 % #1: prefix, #2...#4: selectable languages, #5: base name
226 \NewDocumentCommand{\olsMakeTrilingualMacro}{0{\ols_sprefix} m m m o}{
227   \str_if_eq:eeT{\olsTargetLanguage}{#2} {
228     \IfValueTF{#4}{
229       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#5}{s +m +m +m}{
230         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
231         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##2}}
232         \UseHook{langselect/language/after}
233       }
234     }{
235       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3#4}{s +m +m +m}{
236         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
237         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##2}}
238         \UseHook{langselect/language/after}
239       }
240     }
241   }

```

6 Implementation

```

242 \str_if_eq:eeT{\olsTargetLanguage}{#3} {
243   \IfValueTF{#4}{
244     \ExpandArgs{c}\NewExpandableDocumentCommand{#1#5}{s +m +m +m}{
245       \UseHook{langselect/language/before}{\ols_mapped_target_lang}
246       \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##3}}
247       \UseHook{langselect/language/after}
248     }
249   }{
250     \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3#4}{s +m +m +m}{
251       \UseHook{langselect/language}{\ols_mapped_target_lang}
252       \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##3}}
253       \UseHook{langselect/language/after}
254     }
255   }
256 }
257 \str_if_eq:eeT{\olsTargetLanguage}{#4} {
258   \IfValueTF{#4}{
259     \ExpandArgs{c}\NewExpandableDocumentCommand{#1#5}{s +m +m +m}{
260       \UseHook{langselect/language}{\ols_mapped_target_lang}
261       \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##4}}
262       \UseHook{langselect/language/after}
263     }
264   }{
265     \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3#4}{s +m +m +m}{
266       \UseHook{langselect/language}{\ols_mapped_target_lang}
267       \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##4}}
268       \UseHook{langselect/language/after}
269     }
270   }
271 }
272 }

```

Falls die automatische Erzeugung nicht deaktiviert ist, werden die konkreten Sprachmakros angelegt. Der Name sind $\langle \text{Prefix} \rangle \langle \text{Sprache}_1 \rangle \langle \text{Sprache}_2 \rangle$ bzw. $\langle \text{Prefix} \rangle \langle \text{Sprache}_1 \rangle \langle \text{Sprache}_2 \rangle \langle \text{Sprache}_2 \rangle$.

```

273 \bool_if:cTF{ols_generate}{
274   \bool_if:cTF{ols_trilang}{
275     \olsMakeTrilingualMacro{\olsFrstLanguage}{\olsScndLanguage}{\olsThrdLanguage}
276   }{
277     \olsMakeBilingualMacro{\olsFrstLanguage}{\olsScndLanguage}
278   }
279 }{
280   \PackageInfoNoLine{\packagename}{No~language~macros~generated.\MessageBreak
281   Use~\string\olsMakeBilingualMacro~or~\string\olsMakeTrilingualMacro~to~generate.}
282 }

```

Für die Sternvarianten der Sprachmakros legen wir uns ein Makro an, dass die folgenden Argumente verarbeitet.

```

283
284 \seq_new:N \ols_args_seq
285 \int_new:N \ols_num_of_args
286 \tl_new:N \ols_format_str_tl
287

```

Da das Platzhalterzeichen @ im folgendem Code als Literal auftritt, muss es den gleichen Catcode wie im Dokument haben.

```

288 \makeatother
289 \NewDocumentCommand \olsProcessArgs { +m } {
290   \tl_gset:Nn \ols_format_str_tl{#1}
291   \int_zero:N \ols_num_of_args

```

Suche den höchsten Wert für den @m Formatstring vorkommt. `langselect/language/aftern`

```

292   \int_step_inline:nnnn { 9 } { -1 } { 1 }
293   {
294     \regex_if_match:nnT {@##1}{ #1 }
295     {
296       \int_set:Nn \ols_num_of_args { ##1 } \prg_break:
297     }
298   }

```

Wenn kein Platzhalterzeichen gefunden wurde, gib den Text in Originalform aus. Ansonsten wird das Ersetzungsmakro gerufen.

```

299   \int_compare:nNnTF {\ols_num_of_args} = { 0 }
300   { #1 }{
301     \seq_clear:N \ols_args_seq
302     \ols_collect_replace:n \ols_num_of_args
303   }
304 }

```

rhält *ein* Argument übergeben, nämlich die Anzahl der verbleibenden Ersetzungstexte. Es konsumiert aber *zwei* Argumente. Dieses zweite holt es sich aus dem nach dem Makro liegenden Tokenstrom, wo es den jeweilig nächsten Ersetzungstext findet. Dieser wird in einer Tokensequent gespeichert und `rneut` aufgerufen. `langselect/language/after\ols_collect_replace`

```

305 \cs_new_protected:Npn \ols_collect_replace:n #1 #2{
306   \IfHookEmptyTF{langselect/argument}{
307     \seq_put_right:Nn \ols_args_seq { #2 }
308   }{
309     \seq_put_right:Nn \ols_args_seq {

```

6 Implementation

```

310     \UseHook{langselect/argument}{\ols_mapped_target_lang}{#2}
311   }
312 }
313 \int_compare:nNnTF { #1 } = { 1 }
314 {

```

Wenn alle Ersetzungstexte eingelesen wurden, werden sie nacheinander auf die Platzhalterausdrücke angewendet.

```

315   \int_step_inline:nn { \ols_num_of_args }
316   {
317     \tl_analysis_log:n{@}
318     \tl_analysis_log:n{\ols_format_str_tl}
319     \tl_replace_all:Nnn \ols_format_str_tl
320     { @##1 }
321     { \seq_item:Nn \ols_args_seq { ##1 } }
322   }
323   \ols_format_str_tl
324 }{
325   \ols_collect_replace:n { \int_eval:n { #1 - 1 } }
326 }
327 }
328 \makeatletter

```

Wenn gewünscht, wird Babel oder Polyglossia geladen und die Zielsprache als Hauptsprache gesetzt.

```

329 \bool_if:NT \ols_load_babel{
330   \RequirePackage[main=\ols_mapped_target_lang, provide*=*]{babel}
331   \bool_if:nT{
332     \ols_shorthands && !\str_if_eq_p:nn{\olsTargetLanguage}{de}}
333   {
334     \languageshorthands{ngerman}
335   }
336 }
337 \bool_if:NT\ols_load_polyglossia{
338   \RequirePackage{polyglossia}
339 }

```

Unabhängig davon, wann ein Sprachpaket geladen wurde, wird versucht, die Zielsprache zu aktivieren.

```

340 \cs_if_exist:cT{ver@babel.sty}{
341   \AtBeginDocument{
342     \selectlanguage{\ols_mapped_target_lang}
343   }

```

6 Implementation

```
344 }
345 \cs_if_exist:cT{ver@polyglossia.sty}{
346   \bool_if:NTF\ols_shorthands{
347     \setdefaultlanguage[babelshorthands=true]{\ols_mapped_target_lang}
348   }{
349     \setdefaultlanguage{\ols_mapped_target_lang}
350   }
351 }
352 \bool_if:NT\ols_shorthands {
353   \RequirePackage[autostyle=true]{csquotes}
354   \AtBeginDocument{
355     \cs_if_exist:cT{usesshorthands}{
356       \usesshorthands*{"}
357       \defineshorthand{"`"}{\openautoquote}
358       \defineshorthand{"'"}{\closeautoquote}
359       \defineshorthand{"{/"}{\textormath{\bbl@allowhyphens
360         \discretionary{/}{}{/}\bbl@allowhyphens}{}}
361     }
362   }
363 }
```

Index

`auto`, 5, 6

`babel` (Paket), 12, 13

`bicaption` (Paket), 3

`\closeautoquote`, 9

`comment` (Paket), 3

`csquotes` (Paket), 6, 9

CTAN, 3, 4, 9

`\DocumentMetadata`, 2, 5, 13, 15

EIJKHOUT, Victor, 3

`\enquote`, 9

`fontenc` (Paket), 10

`fontspec` (Paket), 10

`\foreignlanguage`, 4, 9

GREWE, Richard, 3

`\include`, 2

`\label`, 7

`languages`, 5, 11

`load babel`, 5, 9

`load polyglossia`, 9

`\makeatletter`, 10

`\makeatother`, 10

`map`, 6

`multilang` (Paket), 3

`\ols_collect_replace`, 19

`\ols_mapped_target_lang`, 15

`\olsIsoTags`, 10

`\olsMakeBilingualMacro`, 8, 16

`\olsMakeTrilingualMacro`, 8, 16

`\olsTargetLanguage`, 2, 4, 5, 9, 11, 13

`\openautoquote`, 9

`osglecture` (Paket), 9

`polyglossia` (Paket), 12, 13

`prefix`, 5, 8

`\section`, 7

`\selectlanguage`, 4

`targetlang`, 4, 5, 9, 11, 13

`translations` (Paket), 3

`translator` (Paket), 3

`trim`, 5

`unified shorthands`, 6, 9

`\usepackage`, 2, 4

`\varsfromjobname`, 11

`varsfromjobname` (Paket), 4

`xt_captions` (Paket), 3